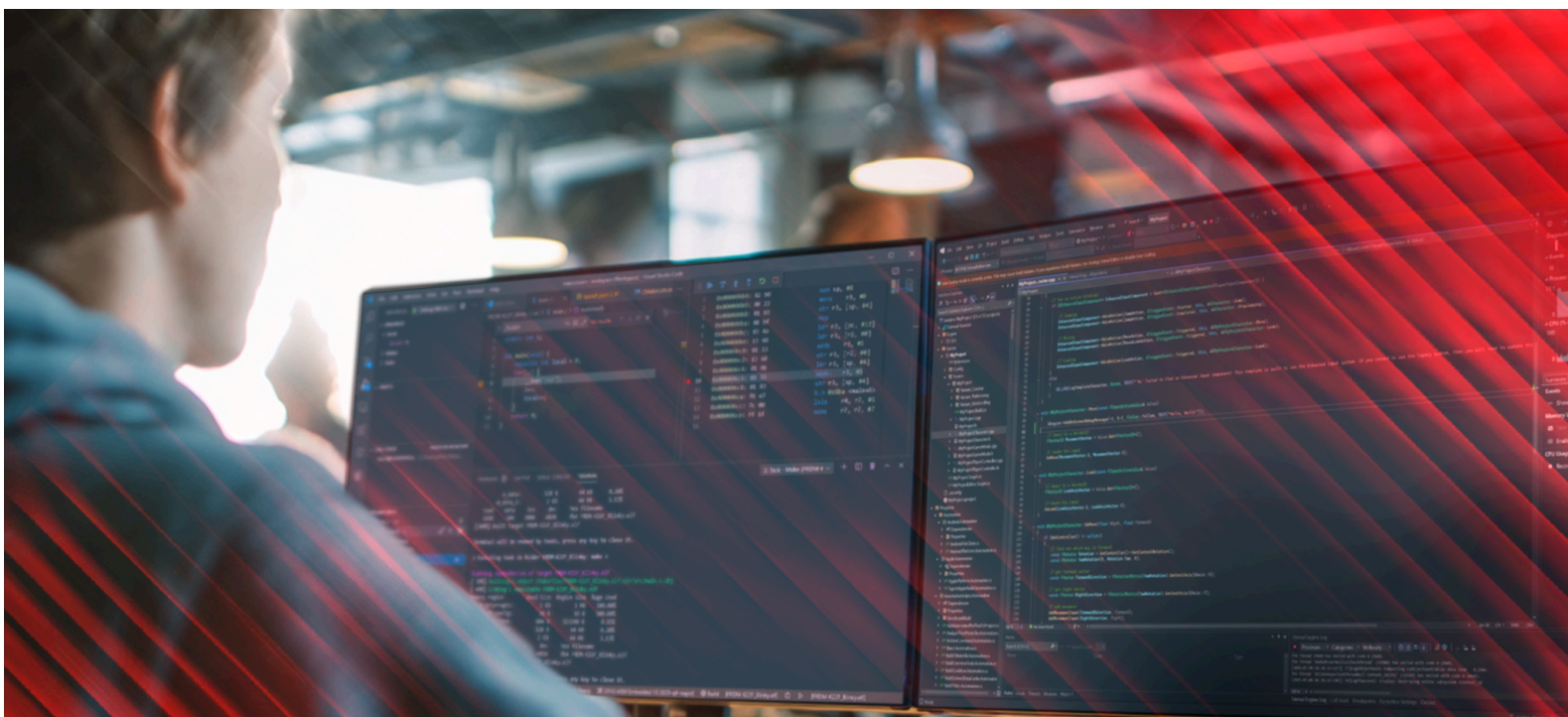




# ТЕСТОВОЕ ЗАДАНИЕ

## Оглавление

|                              |   |
|------------------------------|---|
| Введение                     | 2 |
| Описание правил «Игра Жизнь» | 2 |
| Входные и выходные данные    | 2 |
| Примеры тестовых данных      | 4 |
| Критерии приемки             | 5 |
| Ожидаемые материалы          | 5 |
| Дополнительное задание*      | 6 |
| Заключение                   | 7 |

# Введение

Задание предназначено для оценки навыков в области структур данных, алгоритмизации и проектирования архитектуры программных решений.

В рамках задания необходимо разработать приложение, реализующее клеточный автомат «Игра Жизнь».

## Технические требования:

- **Язык:** C++17 или новее
- **Платформа:** x64 (рекомендуется)
- **Тип приложения:** На усмотрение исполнителя (консольное, UI, игровой движок и т.д.)

Исполнитель самостоятельно выбирает технологии и подход к реализации.

## Описание правил «Игра Жизнь»

Для каждой клетки подсчитывается число живых соседей в окрестности Мура (8 клеток: по горизонтали, вертикали и диагоналям).

### Правила эволюции:

- **Живая клетка:** остаётся живой при 2 или 3 соседях, иначе умирает
- **Мёртвая клетка:** становится живой при ровно 3 соседях, иначе остаётся мёртвой
- **Размер поля:** произвольный

Все клетки обновляются синхронно на основе состояния предыдущего шага (двойная буферизация).

## Входные и выходные данные

Файлы данных располагаются в рабочей директории приложения:

**1. Входной файл:** `game_of_life_input.json`

Считывается при старте игры и выставляет параметры по умолчанию.

**2. Выходной файл:** `game_of_life_output.json`

Создается после принудительной остановки игры или после смерти всех живых клеток.

## Формат входных данных

JSON

```
{
  "world": {
    "width": 5,
    "height": 4,
    "cells": [
      ". * . . .",
      ". . * * .",
      ". . * . .",
      ". . . . ."
    ]
  },
  "simulation": {
    "fps": 30,
    "cycles": 100
  }
}
```

### Описание полей

`world.height` — высота поля (целое число > 0).

`world.width` — ширина поля (целое число > 0).

`world.cells` — начальное состояние: массив из `height` строк длиной `width`.

'\*' — живая клетка.

'.' — мёртвая клетка.

`simulation.fps` — частота кадров (целое число > 0, использование на усмотрение исполнителя).

`simulation.cycles` — количество итераций эволюции (целое число ≥ 0).

**Граничные условия:** Все клетки за пределами поля считаются мёртвыми.

## Формат выходных данных

JSON

```
{
  "result": {
    "width": 5,
    "height": 4,
    "cells": [
      ". . . . .",
      ". . * . .",
      ". . . * .",
      ". . * * ."
    ]
  }
}
```

Значения `height` и `width` должны совпадать с входными.

Массив `cells` содержит состояние мира на момент остановки после `cycles` итераций.

## Примеры тестовых данных

### Тест 1: Осциллятор «Мигалка»

JSON

```
{ "world": { "height": 5, "width": 5, "cells": [ ".....",  
  "...*...", "...*...", "...*...", "....." ] }, "simulation": { "fps": 1,  
  "cycles": 1 } }
```

Ожидаемый выход:

JSON

```
{ "result": { "height": 5, "width": 5, "cells": [ ".....",  
  ".....", ".***.", ".....", "....." ] } }
```

### Тест 2: Пустой мир

JSON

```
{ "world": { "height": 3, "width": 3, "cells": [ "...", "...",  
  "..."] }, "simulation": { "fps": 1, "cycles": 10 } }
```

Ожидаемый выход:

JSON

```
{ "result": { "height": 3, "width": 3, "cells": [ "...", "...",  
  "..."] } }
```

### Тест 3: Стабильная конфигурация «Блок»

JSON

```
{ "world": { "height": 4, "width": 4, "cells": [ ".....", ".**...",  
  ".**...", "....." ] }, "simulation": { "fps": 60, "cycles": 100 } }
```

Ожидаемый выход (без изменений):

JSON

```
{ "result": { "height": 4, "width": 4, "cells": [ ".....",  
  ".**...", ".**...", "....." ] } }
```

# Критерии приемки

Готовая работа будет оцениваться по следующим критериям:

- Проект собирается из переданных исходников
- Приложение корректно читает `game_of_life_input.json`
- Корректная реализация правил «Игра Жизнь»
- Корректная запись результата в `game_of_life_output.json`
- Успешное прохождение тестовых примеров
- Архив содержит только исходники без бинарников
- Присутствует README.md с инструкциями
- Оптимальный объем кода
- Срок выполнения

## Ожидаемые материалы

По результату выполнения тестового задания ожидается архив в формате .zip:

**1. Исходный код** — все необходимые файлы проекта

**2. README.md** с описанием:

- Используемая технология/платформа
- Инструкция по сборке
- Инструкция по запуску
- Используемые библиотеки (при наличии)

**3. Пример входного файла** `game_of_life_input.json`

Что НЕ включать в архив:

- Бинарные файлы (`.exe`, `.dll`, `.so`)
- Промежуточные файлы сборки (`.obj`, `.o`, `.pdb`)
- Папки сборки (`bin/`, `obj/`, `Debug/`, `Release/`, `x64/`, `Intermediate/`, `Binaries/`, `Library/`, `Temp/`)
- Кэш и временные файлы IDE (`.vs/`, `.suo`, `.user`, `.VC.db`)
- Папки зависимостей (`packages/`, `node_modules/`)
- Любые файлы, увеличивающие размер архива без необходимости

## 4. Сопроводительное письмо

Для более точной оценки выполненного задания необходимо в свободной форме подготовить **КРАТКОЕ** сопроводительное письмо и предоставить его вместе с выполненным техническим заданием.

В письме необходимо описать:

- Общий подход к решению задачи и выбранную архитектуру;
- Какие части системы и каким образом были реализованы;
- Используемые библиотеки, фреймворки, игровые движки, сторонние сервисы, сайты, AI инструменты или агенты (при наличии);
- Возможные ограничения, упрощения или компромиссы, принятые в процессе разработки;
- Любые дополнительные нюансы, важные для понимания решения.

## Дополнительное задание\*

Предлагается, по вашему желанию, предоставить решение дополнительной задачи, по добавлению в уже созданный вами проект (основное задание выше) «геймплейной фичи».

Продюсерская фича — «**Авиаудар**». Аналогичную можно посмотреть во многих игровых проектах.

Условия использования авиаудара - уничтожает все живые клетки в заданном радиусе в указанной зоне (для «Игра Жизнь»).

В удобном для вас формате, оцените реализацию добавления данной «фичи» (этапы работ, задачи, ориентировочные сроки выполнения) и постарайтесь сделать один или несколько пунктов (на ваш выбор):

1. Опишите ход мыслей, как бы Вы реализовали/интегрировали такую фичу (набор задач) в текущий (ваш) проект.
2. Обозначьте возможные геймплейные, визуальные, технические и другие ограничения, относительно реализации и использования данной фичи.
3. Предоставьте конфигурацию настройки параметров фичи (желательно json-файл с описанием параметров и ограничений значений) для балансировки геймдизайнерами.

*\*При возможности, добавьте эту фичу в код вашей игры, согласно описанного вами выше.*

## Заключение

При возникновении вопросов, напишите нам, мы готовы на них ответить. Результат выполненного тестового задания используется компанией только в рамках оценки навыков соискателя.

Желаем удачи и надеемся, что выполнение тестового задания будет увлекательным и необременительным.

С уважением,  
Команда **IgroTek**.

